

TCS BFSI Garaje Hackathon Prototype Report

Name: Sri Satya Narendra Yalla

Reference no.: CT20244483953

Chosen problem no: 3

3. Problem Statement: AI-Powered Insurance Policy Information Chatbot

Background: Insurance companies offer a variety of policies, including health, life, auto, and home insurance. Customers often have questions about the different types of policies, coverage options, premiums, and claim processes. Providing timely and accurate information is crucial for customer satisfaction and decision-making. An AI-powered chatbot can assist customers by answering their queries about insurance policies, helping them understand their options and make informed decisions.

Objective: Develop a chatbot that can assist customers with queries related to different types of insurance policies offered by the company. The chatbot should understand natural language, provide accurate responses, and guide customers through the information they need.

Requirements:

LLM:

Use LLM for natural language processing

Knowledge Base Integration:

Create a knowledge base with information about different insurance policies, coverage options, premiums, and claim processes. Details can be shared in the form of PDF to create the knowledge base.

Ensure the chatbot can retrieve and present relevant information from the knowledge base.

Conversation Management:

Implement fallback mechanisms to escalate complex issues to human agents if necessary.

User Interface:

Design a simple chatbot interface that can be integrated into the company's website or mobile app.

Presentation:

Prepare a comprehensive report detailing the methodology, results, and conclusions. Explain why the implemented approach was selected.

You may use streamlit for UI. Submit the recording of the demo with voice over of what has been achieved along with the code.

1. INTRODUCTION

Insurance companies offer a wide range of policies—health, life, auto, and home—each with unique terms, coverage options, and claim procedures. Customers often struggle to understand these policies and need quick, accurate assistance to make informed decisions. This project introduces an **AI-powered Insurance Policy Information Chatbot** designed to simplify user interactions by answering queries related to insurance policies. By leveraging natural language processing, semantic search, and a pre-trained language model, the chatbot provides accurate and context-aware responses in real time. It helps improve customer support efficiency and enhances user experience through instant, reliable assistance.

2. OBJECTIVES

The primary objective of this project is to develop an AI-powered chatbot that can assist users in understanding various insurance policies, including health, life, auto, and home insurance. The chatbot is designed to:

- Understand and respond to user queries in natural language.
- Retrieve accurate information from a structured knowledge base.
- Provide clear explanations of policy types, coverage options, premiums, and claim processes.
- Enhance customer support by offering instant assistance.
- Escalate complex queries to human agents when necessary.

The goal is to streamline customer interactions, reduce response times, and improve decision-making through an intelligent, user-friendly interface.

3. METHODOLOGY

The chatbot was developed using a modular approach, combining various tools and technologies to ensure accurate, real-time responses and seamless user interaction. The core methodology involves the following key components:

3.1 Data Collection and Knowledge Base Creation

A domain-specific knowledge base was prepared by compiling over 100 frequently asked questions and answers related to insurance policies, coverage, premiums, and claim processes. The data was curated and structured into a PDF document, which serves as the foundational source of information for the chatbot.

3.2 Text Preprocessing and Chunking

The content from the PDF was extracted and divided into manageable chunks using a text-splitting technique. This ensures that each chunk contains a coherent piece of information suitable for retrieval during the question-answering process.

3.3 Semantic Embedding and Vector Storage

Each text chunk was converted into semantic vectors using the all-MiniLM-L6-v2 model from the Sentence Transformers library. These embeddings capture the contextual meaning of each chunk and were stored in a Pinecone vector database for fast and efficient similarity search.

3.4 Retrieval-Augmented Generation (RAG)

A Retrieval-Augmented Generation pipeline was implemented using LangChain. When a user asks a question, the system first retrieves the top relevant chunks from the vector store based on semantic similarity. These chunks are then passed along with the user query to a language model (OpenAI's GPT), which generates a context-aware and natural response. This combination enhances the accuracy and relevance of the answers provided.

3.5 Web Application Interface

A simple yet functional web interface was built using Flask. The frontend captures user inputs and displays chatbot responses in real time, enabling a smooth and interactive user experience. The application is designed to be easily deployable on both web and mobile platforms.

3.6 Environment and Configuration

Environment variables and API keys (OpenAI and Pinecone) were securely managed using .env files and Python's dotenv module. The app was run on a local server using Flask with the option to scale or deploy to the cloud as needed.

4. WHY IMPLEMENTED APPROACH WAS SELECTED?

In developing an AI-powered chatbot for insurance-related queries, the selected approach was carefully chosen based on a combination of accuracy, scalability, ease of integration, and maintainability.

1 Retrieval-Augmented Generation (RAG) for Contextual Accuracy

Rather than relying solely on a generative model that could "hallucinate" answers, we employed a Retrieval-Augmented Generation (RAG) strategy. This hybrid approach ensures:

- Factual accuracy by grounding responses in a curated knowledge base.
- Flexibility in handling a wide variety of insurance-related questions.
- Minimization of incorrect or misleading outputs, which is critical in a domain like insurance.

Compared to a purely generative model or a rule-based chatbot, RAG offers better contextual understanding and traceability.

2 Use of Pinecone for Vector Similarity Search

For efficient and scalable retrieval, Pinecone was chosen as the vector database. Alternatives like FAISS or Elasticsearch were considered, but Pinecone stood out because:

- It offers serverless vector indexing, which reduces infrastructure overhead.
- It supports real-time updates and retrieval, ideal for dynamic or expanding knowledge bases.
- It integrates seamlessly with LangChain and other modern NLP tools.

3 MiniLM Embeddings for Balance Between Speed and Accuracy

We chose the MiniLM-L6-v2 transformer model for embedding generation due to its:

- High performance on semantic similarity tasks.
- Lightweight nature, ensuring faster processing without compromising on quality.
- Suitability for projects requiring both accuracy and efficiency in a constrained environment.

Heavier models like BERT or Sentence-BERT, while powerful, were not as optimal for our web-based use case with limited compute resources.

4 LangChain for Modular Pipeline Building

LangChain was selected for chaining the retrieval and generation steps because:

- It abstracts complex logic into clean, maintainable chains.
- It integrates easily with LLM providers, vector stores, and prompt templates.
- It supports scalability and extensibility, making future enhancements like feedback loops or multi-turn conversations easier to implement.

5 Flask and HTML for Simplicity and Portability

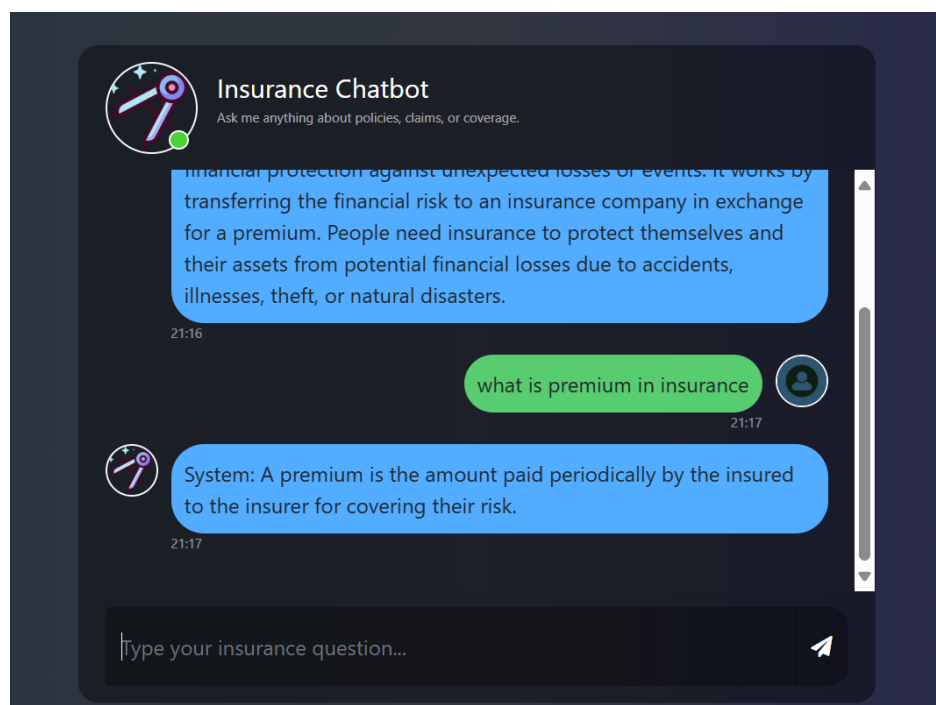
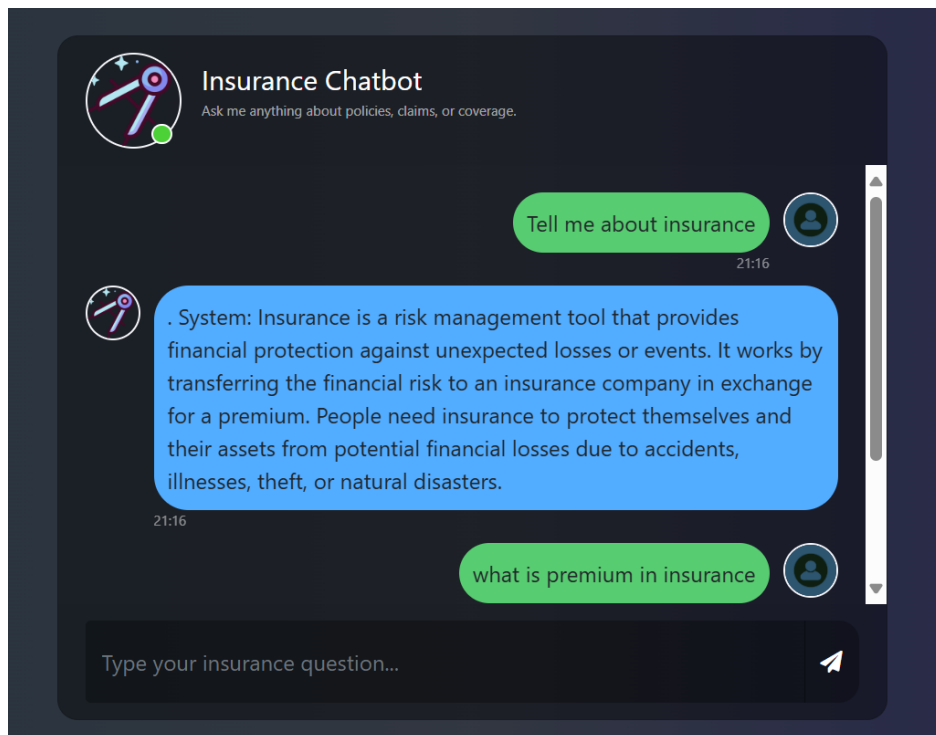
To deliver a proof-of-concept quickly and effectively, Flask was used to build the backend API, with a lightweight HTML frontend. This allowed:

- Rapid prototyping and testing.
- Easy deployment on local servers or cloud environments.
- Integration possibilities with more complex frontends or mobile apps in the future.

Summary

The combination of RAG, Pinecone, MiniLM embeddings, LangChain, and Flask ensures a reliable, accurate, scalable, and easy-to-use solution. Compared to traditional approaches like static FAQ systems or rigid decision trees, this AI-powered chatbot is much better suited to handle real-world, dynamic queries in the insurance domain.

5. RESULTS



6. CONCLUSION

This project successfully demonstrates the implementation of an AI-powered chatbot designed to assist users with insurance-related queries. By integrating natural language processing, semantic search, and a user-friendly interface, the system provides accurate, real-time responses across various insurance domains including health, life, auto, and home.

The use of a structured knowledge base and Retrieval-Augmented Generation (RAG) ensures the chatbot delivers contextually relevant answers while minimizing confusion for end-users. The solution is scalable, easy to deploy, and significantly reduces the dependency on human support for routine inquiries.

Overall, the chatbot enhances customer engagement, improves accessibility to insurance information, and represents a practical application of AI in the insurance sector.